Penetration Test Report — Client A, a law firm

> **Anonymization note.** This report documents real work performed under authorization from the owner. The client's name and infrastructure identifiers were replaced with placeholders (`Client A` , `cliente-a`) before publication. Belona Cybersecurity does not publish identifiable client data without written authorization.

- **Date:** July 2, 2026
- **Type:** Grey-box, authorized by the owner
- **Scope:** `https://cliente-a.web.app` + Firebase backend (Firestore, Storage, Cloud Functions, Authentication). Only `cliente-a` assets.
- **Methodology:** Reconnaissance → authorization testing (anonymous and authenticated) → privilege escalation → remediation → reporting.

Executive Summary

Identified **1 critical**, **1 medium**, and **1 low** severity vulnerability, **all remediated during the engagement**. The core authorization controls (Firestore & Storage RBAC, App Check on Cloud Functions, admin-function protection) **withstood every test** of unauthorized access and privilege escalation. Overall security posture: **solid**. Residual risk: **low**.

Findings

ID	SEVERITY	FINDING	STATUS
C-01	Critical	Public `.git` directory → source code, history, and GitHub token exposed	Remediated
M-01	Medium	Server-side source code exposed (`functions/index.js` , `package.json`)	Remediated
B-01	Low	Missing HTTP security headers	Remediated

C-01 — `.git` exposure + credential leak (Critical)

- **Description:** Firebase Hosting served the entire project folder (`"public": "."`) and the ignore filter did not cover files inside `.git/` . As a result, `/.git/config` , `/.git/index` , `/.git/logs/HEAD` , refs, and objects were publicly accessible, allowing full reconstruction of the private repository. Additionally, `.git/config` contained a GitHub token (`http.extraheader` , `ghs_` type).
- **Impact:** Leakage of private source code, full history, and a GitHub access credential.
- **Evidence:** `GET /.git/config` → 200 with `AUTHORIZATION: basic [redacted]` .
- **Remediation:** Excluded `.git/**` , `functions/**` , `.github/**` , `package.json` from Hosting and redeployed. Verified: resources now return 404.

- **Mitigating factors:** The token was short-lived (`ghs_`) and the local config already used Git Credential Manager (no embedded token).

M-01 — Server-side source exposure (Medium)

`functions/index.js` and `package.json` were publicly served. **Remediated** (excluded from Hosting; 404).

B-01 — Missing security headers (Low)

Only HSTS was present. Added: `X-Frame-Options: SAMEORIGIN` , `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy` , and `Content-Security-Policy: frame-ancestors 'self'` . **Remediated.**

Validated Controls (effective)

- **Firestore RBAC:** Anonymous users and an authenticated `editor` **cannot** read `enquiries` , `case-files` , or `audit-log` .
- **Write segregation by role:** `editor` can only write to `posts` (blog); `guides` and `services` denied.
- **Privilege escalation:** An authenticated `editor` **cannot** grant themselves the admin role (`assign-role` blocked) or list users.
- **App Check:** Cloud Functions reject calls without a valid attestation.
- **Storage:** Case documents are inaccessible (list/download) without an authorized role.
- **MFA:** Accounts require a second factor (TOTP).

Recommendations (future hardening)

1. Strict script CSP (requires refactoring inline scripts to nonces/hashes).
2. Update vulnerable dependencies (`firebase-admin` / `firebase-functions`).
3. Data validation in rules (field types/sizes on `enquiries` / `procesos`).
4. Serve from a dedicated folder (`public/`) instead of the repo root (allowlist vs blocklist).
5. Periodic secret rotation and cost/abuse monitoring.

Conclusion

The architecture correctly applies **least privilege** and **defense in depth**. The critical vulnerability was a **deployment configuration** issue (not application logic) and was fixed during the engagement. Re-testing is recommended after major changes, along with the hardening recommendations.

Engagement performed on owned assets with authorization.

From the technical portfolio of Belona Cybersecurity LLC — Powder Springs, Georgia · belonacyber.com · contact@belonacyber.com

© 2026 Belona Cybersecurity LLC. All rights reserved. Published for evaluation by prospective clients and employers; see `LICENSE` for terms.